Check for updates

ACM DIGITAL LIBRARY

Association for Computing Machinery

acm open

Latest updates: https://dl.acm.org/doi/10.1145/3669940.3707243

RESEARCH-ARTICLE

# RTL Verification for Secure Speculation Using Contract Shadow Logic

**QINHAN TAN**, Princeton University, Princeton, NJ, United States

**YUHENG YANG**, Massachusetts Institute of Technology, Cambridge, MA, United States

**THOMAS BOURGEAT**, EPFL, Lausanne, Switzerland

**SHARAD MALIK**, Princeton University, Princeton, NJ, United States

**MENGJIA YAN**, Massachusetts Institute of Technology, Cambridge, MA, United States

**Open Access Support** provided by:

**EPFL**

**Massachusetts Institute of Technology**

**Princeton University**

**PDF Download**
**3669940.3707243.pdf**
**04 March 2026**
**Total Citations:** 2
**Total Downloads:** 3407

**Published:** 30 March 2025

**Citation in BibTeX format**

ASPLOS '25: 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems
*March 30 - April 3, 2025*
*Rotterdam, Netherlands*

**Conference Sponsors:**
SIGARCH
SIGPLAN
SIGOPS

ASPLOS '25: Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (March 2025)
https://doi.org/10.1145/3669940.3707243
ISBN: 9798400706981

# RTL Verification for Secure Speculation Using Contract Shadow Logic

Qinhan Tan*
qinhant@princeton.edu
Princeton University
Princeton, New Jersey, USA

Yuheng Yang*
yuhengy@mit.edu
Massachusetts Institute of Technology
Cambridge, Massachusett, USA

Thomas Bourgeat
thomas.bourgeat@epfl.ch
École Polytechnique Fédérale de Lausanne
Lausanne, Switzerland

Sharad Malik
sharad@princeton.edu
Princeton University
Princeton, New Jersey, USA

Mengjia Yan
mengjiay@mit.edu
Massachusetts Institute of Technology
Cambridge, Massachusett, USA

## Abstract

Modern out-of-order processors face speculative execution attacks. Despite various proposed software and hardware mitigations to prevent such attacks, new attacks keep arising from unknown vulnerabilities. Thus, a formal and rigorous evaluation of the ability of hardware designs to deal with speculative execution attacks is urgently desired.

This paper proposes a formal verification technique called *Contract Shadow Logic* that can considerably improve RTL verification scalability with little manual effort while being applicable to different defense mechanisms. In this technique, we leverage computer architecture design insights to improve verification performance for checking security properties formulated as software-hardware contracts for secure speculation. Our verification scheme is accessible to computer architects and requires minimal formal-method expertise.

We evaluate our technique on multiple RTL designs, including three out-of-order processors. The experimental results demonstrate that our technique exhibits a significant advantage in finding attacks on insecure designs and deriving complete proofs on secure designs, when compared to the baseline and two state-of-the-art verification schemes, LEAVE and UPEC.

***CCS Concepts:*** • **Security and privacy → Side-channel analysis and countermeasures**; **Logic and verification**.

***Keywords:*** Speculative Execution Attacks, Hardware-Software Contract, Formal Verification, Shadow Logic

*Two co-first authors contributed equally to this research.

This work is licensed under a Creative Commons Attribution International 4.0 License.

*ASPLOS '25, March 30–April 3, 2025, Rotterdam, Netherlands*
© 2025 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-0698-1/25/03
https://doi.org/10.1145/3669940.3707243

**ACM Reference Format:**
Qinhan Tan, Yuheng Yang, Thomas Bourgeat, Sharad Malik, and Mengjia Yan. 2025. RTL Verification for Secure Speculation Using Contract Shadow Logic. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25), March 30–April 3, 2025, Rotterdam, Netherlands.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3669940.3707243

## 1 Introduction

Speculative execution in modern out-of-order (OoO) processors has given rise to various speculative execution attacks, including Spectre [39], Meltdown [42], L1TF [34], and LVI [63]. These attacks exploit the microarchitectural side effects of transient instructions, i.e., the instructions that are executed speculatively due to misspeculation and squashed later. Prior work has demonstrated that speculative execution attacks can leak secrets via a wide range of microarchitectural structures, including caches [38–40], TLBs [27], branch predictors [17] and functional units [8].

Even worse, as the microarchitecture becomes ever more complex, anticipating speculative execution vulnerabilities seems to pose a Sisyphean challenge. For example, Foreshadow [62] identified the L1TF vulnerability, showcasing cross-domain leakage of data residing in the L1 cache. Though the L1TF vulnerability was quickly patched by Intel by flushing L1 data cache upon context switch [34], such mitigation was insufficient to block small variations of the attack, due to the existence of various other structures inside the L1, including the line fill buffer, the store buffer, the load port, etc. These structures were exploited to leak information by a batch of follow-on attacks called Microarchitecture Data Sampling Attacks (MDS) [54, 64]. The cat-and-mouse game described above calls for rigorous approaches to ensure the security of processors.

To this end, this paper aims to develop a formal verification scheme for secure speculation on out-of-order processors working at the *Register-Transfer Level (RTL).* As hardware

design is a lengthy procedure and involves multiple steps, including early-stage high-level modeling and later-stage RTL implementation, we need verification tools at each stage to offer security guarantees. Several prior efforts, such as Pensieve [76], CheckMate [60], and in Guarnieri et al. [28], focus on higher-level abstract models to assist security verification at the early design stage. However, higher-level models are prone to overlooking small but crucial processor structures, like the buffers exploited by the MDS attacks. Thus, we aim to design verification schemes that can operate at the RTL level at a later design stage when all microarchitectural details are in place. We envision our scheme can complement existing early-stage verification schemes to cover the entire hardware design flow.

### 1.1 Blueprint for Systematic and Scalable RTL Security Verification

Existing RTL formal verification schemes for secure speculation pose scalability challenges. Most techniques only work effectively on simple designs such as in-order processors, but struggle to cope with architecturally complex designs like out-of-order processors, leading to timeout. Making a verification scheme work on an out-of-order processor requires the verification engineer to spend significant manual effort to customize security specifications of subcomponents and try to find invariants that help the unbounded proof to go through. This process is very challenging, especially as it requires the engineers to have security expertise and in-depth knowledge of formal methods, computer architecture, and the specifics of the design studied. Moreover, the verification harness that works on one design cannot be easily transferred to another design without requiring new creative insights.

For example, the state-of-the-art verification work UPEC [21, 22, 36] successfully proves several secure speculation scheme on BOOM [80], highlighting a breakthrough along this research direction.[1] However, a careful examination of their verification suite [59] shows that the verification methodology relies on detailed microarchitectural knowledge of the specific processor, as well as expertise in the usage of invariants in obtaining formal proofs. For example, to verify the open-source BOOM [80] processor, they need to write invariants about the reachable states of CSRs, load/-store queues, reorder buffers, caches, etc. The total lines of code to specify the invariants is over 20,000. Moreover, the invariants derived by UPEC are not reusable across different defenses. When verifying STT on BOOM [36], UPEC manually identifies additional invariants related to the bits and logic added by STT compared to its previous version [21, 22].

[1]There are multiple versions of UPEC dealing with different security issues [20, 53]. In this paper, unless otherwise stated, we refer to [21, 22, 36] which tackles speculative vulnerabilities in RTL implementations.

Overall, UPEC requires formal-method expertise and a substantial amount of manual effort for verifying one design, and it has limited reusability given that the manual efforts need to be repaid each time for a new design or design modification.

Alternatively, recent work (LEAVE [69]) proposed to reduce the amount of manual effort needed for RTL verification of secure speculation schemes using automatic invariant search techniques. Those techniques demonstrated great effectiveness on certain in-order pipelined processors, however we found that the approach tends to have a hard time on simple out-of-order processors (Section 7.1).

To summarize, the primary research challenge here is to improve verification scalability to handle more advanced architectures and reduce the manual and non-reusable effort required to complete the RTL security verification.

In this paper, we aim to make progress towards reusable and more scalable verification schemes by leveraging computer architecture insights. Our verification approach brings the computer architects who design the processor into the formal verification loop: we are asking them to design the shadow logic machinery instrumental to our verification task, which is then used by the verification tools to formally check the security property. Compared to writing logical invariants, writing this shadow logic machinery is a task much closer to the standard activity of computer architects. Further, as we will show, the shadow logic is only a few hundred lines of code, compared to the 20,000 lines of code used to specific the invariants in UPEC.

### 1.2 This Paper: Contract Shadow Logic

The security property to be checked in secure speculation is expressed as a software-hardware contract [14, 28], which states, "if a software program satisfies a constraint, then the hardware ensures executing the program will not leak secrets." Checking this statement involves two operations. We refer to the operation that checks the software against a constraint as the **contract constraint check**, and refer to the operation that checks the hardware for detecting information leakage as the **leakage assertion check**. Each of the two checks is performed on a pair of state machines (or machines for short). The contract constraint check compares the executions of two simple machines which implement the ISA and execute each instruction in one-cycle (henceforth referred to as single-cycle machines or ISA machines). Similarly the leakage assertion check compares the executions of two copies of the out-of-order processor to check for secret leakage. The baseline verification scheme examines the *four* state machines cycle by cycle.

We propose a verification scheme, called Contract Shadow Logic. The key insight is that we can *perform both the contract constraint check and the leakage assertion check on a single pair of out-of-order machines*, and thus eliminate the pair of single-cycle machines in the verification scheme. Specifically,

assuming the out-of-order processor is correctly implementing the ISA semantics, the single-cycle machine's execution trace can be recovered from the out-of-order processor's *committed* instruction sequences. We implement this ISA trace extraction mechanism as shadow logic (which is added to assist verification, does not interfere with the original design, and can be safely removed before synthesis).

We have implemented our verification scheme using the commercial RTL verification tool, JasperGold [10] and use it to verify one in-order processor, three out-of-order processors, and six defense augmentations. We provide experimental results to compare our scheme with the baseline and two state-of-the-art verification schemes, LEAVE [69] and UPEC [22, 36]. Through our results, we demonstrate the following: First, across all secure designs, the baseline scheme cannot provide any security proofs within 7 days, while, our scheme can prove the security for all of them except the secure version of BOOM. Second, while LEAVE cannot automatically find bugs or prove security for our in-house simplified out-of-order processor, our new scheme is able to do so. Third, we show the ability of our method to find attacks on the BOOM [80] processor while requiring significantly less manual effort than UPEC. However, we would like to point out that scalability continues to be a challenge in formally verifying secure speculation of hardware designs, and one will need additional architectural insights to assist formal verification to scale to more complex out-of-order processors.

In summary, this paper makes the following contributions:

1. We propose a verification scheme, called Contract Shadow Logic. Our scheme leverages computer architecture insights to improve the scalability of RTL security verification for secure speculation while maintaining broad applicability (Section 4).
2. We identify two important requirements to construct correct shadow logic for checking software-hardware contracts (Section 5).
3. We demonstrate the efficacy of our scheme on four processors and conduct a detailed comparison to showcase the advantages of our scheme compared to the baseline and two state-of-the-art verification schemes, i.e., LEAVE and UPEC (Section 7).

# 2 Background

## 2.1 Speculative Execution Attacks and Defenses

Speculative execution attacks exploit the microarchitectural side effects of transient instructions. Transient instructions can be introduced via a wide range of speculation features, including branch history prediction [38], branch target prediction [39], return address prediction [40, 44], store-to-load forwarding [29], value prediction [65], and 4K aliasing [11]. The side effects of transient instructions are diverse, including modifying the cache states [38–40, 42, 44], TLBs [27], Line Fill Buffer [54], and DRAMs [58].

Speculative execution vulnerabilities can be mitigated via software and hardware methods. In software, existing mitigations typically involve placing `fence` instructions after sensitive branch instructions to delay the execution of the subsequent instructions until the branch is resolved [3, 35]. Other software approaches involve replacing branch instructions with `cmov` instructions and using bit masking of the index used for array lookups [12]. Hardware mitigation mechanisms focus on restricting speculative execution by either selectively delaying the execution of instructions that can potentially transmit secrets via side channels [43, 52, 71, 77] or making it difficult for attackers to observe the microarchitectural side effects of speculative instructions [1, 2, 37, 75].

## 2.2 SW-HW Contracts for Secure Speculation

As speculative execution vulnerabilities can be mitigated via either software or hardware methods, it becomes challenging to cleanly and explicitly reason about security properties a defense should achieve. The community has been using **software-hardware contracts** to define security properties precisely related to speculative execution vulnerabilities.

Guarnieri et al. [28] proposed the first set of formalized contracts by defining which program executions a side channel adversary can distinguish. Specifically, a contract states that: "if a program satisfies a specified **software constraint**, then the hardware needs to achieve a **noninterference** property to ensure no secrets are being leaked." Formally, a contract can be written as follows:

$$\begin{aligned} & \forall P, M_{pub}, M_{sec}, M'_{sec}, \\ \textbf{if} \quad & O_{ISA}(P, M_{pub}, M_{sec}) = O_{ISA}(P, M_{pub}, M'_{sec}) \\ \textbf{then} \quad & O_{\mu Arch}(P, M_{pub}, M_{sec}) = O_{\mu Arch}(P, M_{pub}, M'_{sec}) \end{aligned} \tag{1}$$

The formula states that when running a program $P$ with public memory $M_{pub}$ and secret memory $M_{sec}$ on a microarchitecture processor $\mu Arch$, if the software observation denoted by $O_{ISA}$ for a program $P$ cannot distinguish between different values of $M_{sec}$, i.e., there is no leakage at the program level, then the microarchitecture observation $O_{\mu Arch}$ cannot distinguish between different values of $M_{sec}$, i.e., there is no leakage at the microarchitecture level.

Note that Eq. (1) does not describe a single contract but a family of contracts. The equation is parameterized by different execution modes at the software level and different observation functions. The scheme proposed in this paper supports the *sequential* execution mode where the software constraint considers traces when executing the program sequentially (i.e., not speculatively). For example, under a so-called sandboxing contract, the software constraint is

that executing the program sequentially will not have secrets loaded into the register file. The $O_{ISA}$ of sandboxing includes the data written to registers of every committed load. In contrast, in the constant-time programming contract, the software constraint is that executing the program sequentially will not use secrets as addresses of instruction or data memory accesses or as operands to hardware modules whose timing depends on the input. This matches the constant-time programming discipline [28]. The $O_{ISA}$ of this contract includes the branch condition, memory address and multiplication operands of committed instructions.

The microarchitecture observation ($O_{\mu Arch}$) considered in this paper includes the address sequence at the memory bus and the commit time of every committed instruction, which is commonly used in prior work [49, 76] to reflect cache and timing side channels.

### 2.3 RTL Verification using Model Checking

Model checking is widely used in verifying hardware designs, including processors [7] and accelerators [31]. It is available as both open-source tools (e.g., AVR [26]) and commercial tools (e.g., Cadence Jaspergold [10]) that directly work on RTL designs. These tools are highly automated, where the user only needs to provide the hardware source code instrumented with the property to be checked. Existing tools have flexible support to express various properties via instrumenting hardware code using a design-friendly language, e.g., System Verilog Assertions (SVA). SVA allows specifying (i) assumptions (using the `assume` keyword) that describe system assumptions such as constraints on the inputs and (ii) assertions (using the `assert` keyword) to specify the property that the model checker is required to check, similar to assertions in C/Python.

As model checking effectively considers all possible input sequences which can be of unbounded length, it provides an unbounded-length proof or a complete proof. When the property does not hold at some reachable state, model checking reports an input-sequence trace, a counterexample, for which the property fails. Bounded model checking (BMC) can be used to perform verification for all possible input sequences of length bounded by some user-provided parameter $k$ – this effectively checks the property for all states reachable within $k$ steps from the initial state which is obviously not as strong as an unbounded-length proof.

The information leakage security property is a hyperproperty, i.e., it needs reasoning over two traces (corresponding to two different values of the secret) to prove or disprove it. This can be handled by constructing a model checking instance with two copies of the design (with different values of the secret), and constructing a property comparing traces over the two copies.

### 2.4 Shadow Logic

When verifying complex RTL code, it is often necessary to introduce auxiliary code to assist verification by obtaining or deriving extra information from the current state of the system. Such auxiliary code is called shadow logic (or ghost code in software verification [41]). This logic *monitors* the design and assumes and asserts that certain conditions hold.

For example, consider a verification task that checks whether a processor correctly implements its ISA specification. For every instruction that updates the register file, we need to check that when the instruction commits, the target register holds the value specified by the ISA. However, due to complex renaming logic in out-of-order processors, it is a nontrivial task to determine which register in the physical register file is the target register. Prior work [51] customizes shadow logic according to the renaming logic to correctly locate the target register as part of this checking.

It is important that this shadow logic only *monitors* the design and does *not* make any modifications that change the behavior of the system. It is common practice to implement shadow logic for verification purposes under some macro. This enables this logic to be stripped out during design compilation and thus, shadow logic is not present in the final production RTL.

## 3 Threat Model

Recall from Section 2.2, security properties for secure speculation can be formulated as software-hardware contracts. Such contracts explicitly state the responsibilities of the software and the hardware sides to achieve an overall noninterference property [18]. In this paper, we consider the following threat model.

1. We assume a system whose data memory is split into two parts: *public* and *secret*. An attacker cannot directly access the secret region. The attacker's goal is to infer any secret value through microarchitectural side-channel attacks. Non-digital side-channel attacks, such as power side channels [45, 67], are out of scope.
2. The system runs a program that satisfies certain software constraints. These constraints prevent the secret from being leaked via ISA state. For example, at the ISA level, the program cannot load secret data into registers (in the sandboxing contract), or it needs to follow the constant-time programming discipline.
3. The attacker is able to gain cycle-level observation of certain micro-architectural variables in the system, including the address sequence transmitted on the memory bus and the commit time of instructions.

## 4 Motivation and Insights

In this section, we describe the insights that motivate the design of our verification scheme. We start by presenting a baseline verification scheme for secure speculation and

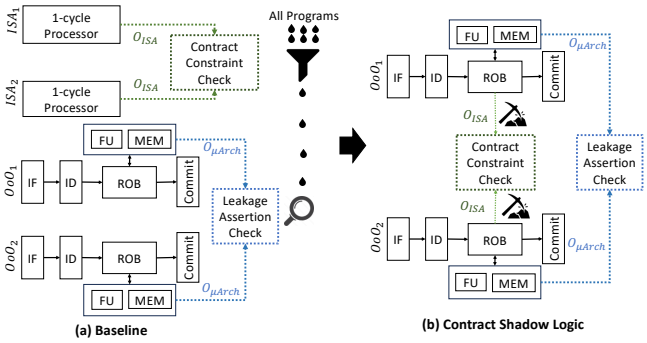


**Figure 1.** Overview of Contract Shadow Logic.

we analyze the performance bottlenecks encountered. We then point out the optimization insight that is architecture-friendly and leads us to the design of Contract Shadow Logic.

### 4.1 The Baseline Verification Scheme

Verifying the software-hardware contract described in Section 2.2 requires performing *two* kinds of equivalence checks that involve *four* traces. The equivalence check at line 2 in Eq. (1) is the *contract constraint check* to determine whether a program $P$ needs to be protected (we say that $P$ is valid). The equivalence check at line 3 is the *leakage assertion check* to determine whether the microarchitecture processor $\mu Arch$ has a correct defense mechanism for the given program.

The security property (Eq. (1)) can be tentatively verified using the scheme visualized in Fig. 1(a). First, two copies of a single-cycle machine are instantiated as two separate RTL modules. The two modules have identical RTL code with the instruction memory and public data memory initialized with the same content ($P$ and $M_{pub}$) and different secret memory ($M_{sec}$), i.e., two secret memories differ in at least one location. The two instances of single-cycle machines execute instructions sequentially, finishing one instruction per cycle. These two copies execute in a lock-step manner, allowing the verification scheme to derive the $O_{ISA}$ trace at each cycle and compare them for equivalence to enforce the contract constraint check. The comparison results serve as a constraint to filter out invalid programs for the out-of-order processor being verified. The model checker will discard the programs that generate the traces violating the constraint.

Similarly, the RTL for the out-of-order processor is duplicated to create two instances, which only differ in the secret memory. The program memory $P$ and the initial value of the public data memory $M_{pub}$ are the same as in the single-cycle machine. The verification scheme compares the microarchitecture execution traces $O_{\mu Arch}$ from the two machines at each cycle for the leakage assertion check.

This scheme runs the contract constraint check and leakage assertion check in parallel at each cycle, allowing for conducting both bounded and unbounded checking.

### 4.2 Architecture-driven Insights for Verification

As discussed in Section 1, the community faces serious challenges in coming up with verification schemes that are both reusable for verifying different defense designs and, meanwhile, scalable enough to be able to verify out-of-order processors. In this paper, we aim to tackle the research challenge of designing a reusable verification scheme while achieving reasonable scalability, where the verification scheme can handle some out-of-order processors without requiring to come up with custom functional invariants. A baseline verification scheme employs four state machines (for the four processors) which contain a non-trivial amount of redundancy among them, we aim to explore the possibility of reducing this redundancy.

As mentioned in Section 1, if an out-of-order processor functionally correctly implements its ISA specification, the ISA trace can be reconstructed from the out-of-order processor's committed instruction sequence. In other words, the single-cycle machine is contained in the out-of-order machine, including the complex instruction decoding, the execution logic, as well as registers/memory state. This insight leads to our verification scheme, introducing shadow logic to implement the contract constraint check directly on the out-of-order processors, as shown in Fig. 1(b). With this, we can reduce a four-machine verification problem (the baseline) to a two-machine problem with some additional shadow logic – this reduction in logic could potentially lead to better scalability.

## 5 Contract Shadow Logic

In this section, we present the main components of Contract Shadow Logic. We first describe the basic component of our shadow logic that extracts the ISA traces from out-of-order processors. We then discuss the challenges in making the shadow logic correctly verify software-hardware contracts, and finally provide our solution adressing these challenges.

### 5.1 Shadow Logic to Extract ISA Traces

An important task of the shadow logic is to monitor the commit stage of the out-of-order processor and extract information for every committed instruction to derive the ISA trace. The shadow logic needs to be implemented according to the observation function ($O_{ISA}$). For example, if checking for the constant-time contract, the contract constraint check needs to compare the addresses of committed memory access instructions and the condition values of committed branch instructions. Different processor implementations may store this information in different places. In the best case, the information is already present in the ROB and can be directly obtained.

In more complex cases, the information may be spread across multiple modules, such as inside the register files and load/store queues, and in some cases the information might

not even be readily available at commit-time anymore. This is the place where we can leverage and involve the architect's knowledge and expertise. Specifically, to handle these cases, the user needs to extend the existing ROB structure with shadow metadata state to record the information needed to reconstruct the ISA trace. For example, instead of extracting this information from across modules, we can record the opcode type with the instruction when it enters the ROB and its operand values when it finishes execution. If the instruction commits later, we can directly obtain the information for the contract constraint check.

Nonetheless, even though the shadow logic needs to be customized for different processor designs, the task of deriving shadow logic to extract ISA traces is straightforward for the processor designers to carry out. We think this presents an important advantage of our scheme compared to related work, as our scheme can get the designers involved in the verification loop and make the formal tools more accessible to architects. Moreover, our verification scheme is reusable across different design variations, defenses and contracts, as we only need to swap in different ISA trace extraction logic without requiring to search for new invariants as needed by existing work.

Remarkably, there's a scenario where no additional engineering effort is required for tackling different designs. Imagine a hardware designer integrating security measures like STT and DoM onto an initially insecure processor. The shadow logic, initially integrated for the insecure design, seamlessly transfers to the secure design without alteration. Indeed, all the necessary signals for the shadow logic remain intact within the processor when adding the defense mechanism. We use this observation in Section 7.2, where we assess different defense schemes employing precisely the same shadow logic.

### 5.2 Challenges: Two Requirements

Enforcing the contract constraint check needs comparing two ISA-level traces. However, naively introducing shadow logic to extract committed instruction information is insufficient to verify software-hardware contracts since it will violate what we refer to as the **instruction inclusion requirement** and the **synchronization requirement**.

#### 5.2.1 Instruction Inclusion Requirement.

Recall from Eq. (1): the contract constraint check and the leakage assertion check examine the same program $P$. This leads to the instruction inclusion requirement: "*if an instruction's microarchitecture side effects have been considered in the leakage assertion check and it will eventually commit, this instruction must also be examined by the contract constraint check*."

The baseline verification scheme assumes the out-of-order processor fetches at most one instruction per cycle and the ISA processor executes one instruction per cycle. The instruction inclusion requirement is automatically fulfilled as the ISA processor is always running ahead of the out-of-order processor.

However, when switching to our optimized verification scheme, enforcing the instruction inclusion requirement requires additional work. Indeed, intuitively, in the out-of-order machines, the contract constraint check is checked at commit time, because the contract constraint check only examines committed instructions. This is a bit behind the leakage assertion check, meaning we may miss the instructions that are still in their execution or even the fetch stage. Thus, we need to make sure that any violation of the leakage assertion check is not related to a failure of contract constraint check that has not occurred yet, i.e., the violation of the leakage assertion check is not for a program that should have been filtered out by the contract constraint check.

**Solution** Our solution to enforce the instruction inclusion requirement is to structure the verification logic into two phases. In the first phase, we perform the contract constraint check and leakage assertion check on a pair of out-of-order processors cycle by cycle. Upon detecting microarchitecture trace deviation, we switch to the second phase, where the contract constraint check is applied to all the inflight and bound-to-commit instructions in the pipeline, satisfying the instruction inclusion requirement.

#### 5.2.2 Synchronization Requirement.

In the baseline verification scheme, for each pair of machines, the two machines in the pair execute in a lock-step manner. Hence, the scheme can perform both the contract constraint check and the leakage assertion check by comparing the corresponding states in each pair at *each cycle*. Because the ISA machine executes exactly one instruction per cycle, the check on ISA traces ($O_{ISA}$) is equivalent to comparing states instruction after instruction.

However, comparing at each cycle the new elements of the ISA traces derived from the two out-of-order processors will not be an equivalent check. This is because the out-of-order machines might run out of synchronization when the two processors commit their instructions at different times. Consider the case when the two processors try to commit a load instruction with different addresses, which can happen because of different values of the secret. One of the processors commits the load in one cycle as the load hits the L1 cache, while the other processor suffers a cache miss and commits the load after the result returns from DRAM, potentially after a hundred cycles. Because the instruction commits at different cycles, the ISA trace derived from the first processor is no longer cycle-aligned with the ISA trace derived from the second processor.

**Solution** To perform the same check for the contract constraint check in the baseline scheme, we first need to *re-align* the derived ISA traces. We achieve this by introducing shadow logic to the second phase of our verification scheme

```
1  cpu cpu1 (.clk(pause1 ? 0: clk), .rst(rst));
2  cpu cpu2 (.clk(pause2 ? 0: clk), .rst(rst));
3
4  always @(posedge clk) begin
5    // shadow logic to compare traces
6    uarch_diff = (O_uarch(cpu1)!=O_uarch(cpu2));
7    isa_diff   = (O_ISA(cpu1)  !=O_ISA(cpu2));
8
9    // phase-1
10   initial uarch_diff_phase1 <= 0;
11   if (uarch_diff_phase1==0 && uarch_diff) then
12     uarch_diff_phase1 <= 1;
13     tail1 <= cpu1.tail;
14     tail2 <= cpu2.tail;
15   endif
16
17   // phase-2
18   if uarch_diff_phase1 then
19     // Requirement 1: inst inclusion
20     drained = (tail1==cpu1.head-1) &&
21               (tail2==cpu2.head-1);
22
23     // Requirement 2: realign ISA traces
24     if (cpu1.commit==cpu2.commit)
25         pause1 <= 0;
26         pause2 <= 0;
27     else if (cpu1.commit)
28         pause1 <= 1;
29     else if (cpu2.commit)
30         pause2 <= 1;
31   endif
32
33   // contract assumption check
34   assume (isa_diff==0);
35   // leakage assertion check
36   assert (!(uarch_diff_phase1 && drained));
37 end
```

**Listing 1.** Verilog-Style Pseudo-code for Shadow Logic.

to synchronize the out-of-order processors to commit instructions at the same speed.

### 5.3 Solution: Two-Phase Shadow Logic

Listing 1 shows the pseudocode of our two-phase shadow logic written in Verilog style. Given an out-of-order processor to be verified, we create two instances of the processor, named cpu1 and cpu2 (lines 1-2). We then introduce shadow logic to drive and monitor the microarchitecture traces and ISA traces of the two instances cycle by cycle (lines 5-7).

The remaining shadow logic uses the trace comparison results and operates differently in two phases. In Phase 1, upon a microarchitecture trace deviation, we record the deviation by setting the variable `uarch_diff_phase1` to true and also record the tail of the ROB of each cpu instance. The goal of recording the tail is to assist the shadow logic in Phase 2 to determine when the contract constraint check is completed to satisfy the instruction inclusion requirement.

In Phase 2, since a microarchitecture trace deviation has already happened, the responsibility of the remaining shadow logic is to ensure the contract constraint check is sound, satisfying both the instruction inclusion requirement and the synchronization requirement. Specifically, the shadow logic tracks whether the two instances have committed the instruction that was recorded at the end of Phase 1 (lines 19-21). Note that the actual shadow logic implementation also accounts for the case when the recorded tail is squashed due to mis-speculation, which we do not show in Listing 1 for brevity. Besides, the shadow logic also re-aligns the ISA observation traces whenever the two instances go out of synchronization. The re-alignment operation is achieved by manipulating the input `clk` signal to the two cpu instances, involving no code modifications inside the cpu code (see lines 23-30 and how `pause` is used in lines 1-2).

Note that the re-alignment operation only happens in Phase 2 because the commit signal is included in the $O_{\mu arch}$ trace, which is a common practice in contracts targeting timing side channels. This example shows a machine (the cpu instance) with one clock domain. In the case that the machine has multiple clock domains, e.g., using different clocks for the processor and caches, all the clocks need to be paused and resumed simultaneously.

Finally, we use SVA to explicitly express the contract constraint check and leakage assertion check in lines 33-36.

**Model Checking with Contract Shadow Logic** Given the cpu state machines and the shadow logic, model-checking techniques check for all possible input states that satisfy the assumption ($M_{pub}$ and $M_{sec}$) and answer the question whether any of the input states can trigger the assertions. We expect three possible outcomes:

1) the model checker finds a counterexample, which reassembles an attack code that satisfies the software constraints but will introduce distinguishable microarchitecture traces running on the cpu;
2) the model checker declares a successful unbounded proof to indicate the processor under verification does satisfy the software-hardware contract for unbounded number of cycles;
3) the model check hits a scalability bottleneck and cannot produce a result within a reasonable amount of time, which we refer to as "time out."

**Validity of the Synchronization Operation** As discussed in Section 2.4, shadow logic should only passively monitor operations and never modify the design under verification. However, the shadow logic we introduce does have a limited influence on the design when the "`pause`" signal freezes the clock. Strictly speaking, this logic is no longer just monitoring the design. We claim our shadow logic is still valid for two reasons. First, the shadow logic performs the "`pause`" operation in Phase 2. At the beginning of Phase 2, the timing-dependent microarchitectural traces deviation has already

been detected, thus Phase 2 only focuses on comparing traces of committed instructions, i.e., ISA traces, which only involve the execution results of committed instructions and does not include any microarchitectural timing information. Second, the "pause" signal only freezes the clock signal of the processor and thus does not affect the execution results of committed instructions. Thus, it will not influence the ISA traces, nor will it affect the ISA trace comparison results.

**Supporting Superscalar Processors** The pseudocode in Listing 1 assumes the processor commits maximally one instruction at a cycle. Our scheme can be adapted for superscalar processors, where multiple instructions can be committed at each cycle. The problem that we need to tackle is that the pausing granularity is determined by how many instructions the processor commits. We will need to add shadow logic to break the atomicity of contract constraint check by supporting partial ISA trace comparison at each cycle and the capability to remember unaligned traces for future comparison. This approach introduces minimal shadow state overhead as the number of entries for memorizing unmatched ISA traces only needs to match the commit bandwidth of the processor. It is constant regardless of the ISA trace length. However, the complexity of the logic increases with the commit bandwidth because we need to align traces across neighboring cycles.

### 5.4 Discussion

**Verification Gap.** Our verification scheme reformulates the contract property in Equation 1 and can potentially introduce a verification gap. Our proof is conditioned upon the ISA traces stated as the hypothesis in Equation 1 matching the ISA traces extracted from the out-of-order processors, which in turn requires the following assumptions to be satisfied: 1) the shadow logic is assumed to be implemented correctly; 2) the out-of-order processor is assumed to be functionally correct.

First, the definition of the correctness of our shadow logic includes the shadow logic selecting the right signals to extract the ISA trace and the microarchitectural trace, and the shadow logic generating the pause signal at the proper time. As such, we ensure the derived ISA trace from the out-of-order processor contains the writeback data of each corresponding instruction.

Second, the functional correctness assumption ensures that the commit stage generates the correct ISA instructions and writeback data as the ISA specification. Together with the shadow logic correctness assumption, we ensure the derived ISA traces in our scheme are equivalent to the ISA hypothesis in the contract property.

**Decoupling of security and functional verification.** Our verification approach leverages the decoupling (i.e., separation) of functional verification and security verification. Specifically, our approach for verifying a security property generates a verification task that does not embody a proof obligation related to the functional correctness of the out-of-order processor. Instead, as stated above, we assume the machine has been independently verified to be functionally correct. Our choice is justified because verification engineers spend significant effort on functional verification, and our approach leverages this and avoids having to redo this effort as a sub-task of security verification, which may explain the significant verification performance improvement obtained with our approach. Similar decoupling approaches are also used in inductive approaches on verifying contract properties such as LEAVE [69].

| | |
|---|---|
| Sodor [15] | Open-source in-order processor<br>ISA: the full suite of RV32I<br>Configuration: 2-stage pipeline, 1-cycle memory<br>restricted to use a 4-entry register file<br>Code size: 700 lines of Chisel (2,700 lines of Verilog)<br>~90 lines of Verilog for shadow logic<br>~3h manual effort for shadow logic |
| Simple OoO | Our in-house minimal out-of-order processor<br>ISA: 4 customized insts (loadimm, ALU, load, branch)<br>Configuration: 4-stage pipeline<br>4-entry ROB, 4-entry register file<br>commit bandwidth is 1 inst/cycle<br>Code size: 1,000 lines of Verilog<br>~100 lines of Verilog for shadow logic<br>~5h manual effort for shadow logic |
| Ride core [33] | Open-source out-of-order processor<br>ISA: 35 instructions in RV32IM<br>Configuration: 6-stage pipeline<br>8-entry ROB, 64 physical registers<br>commit bandwidth is 2 inst/cycle<br>Code size: 8,100 lines of Verilog<br>~400 lines of Verilog for shadow logic<br>~10h manual effort for shadow logic |
| BOOM [80] | Open-source out-of-order processor<br>ISA: RV64GC<br>Configuration: SmallBoomConfig<br>32-entry ROB, 52 physical registers<br>commit bandwidth is 1 inst/cycle<br>Code size: 19k lines of Chisel (136k lines of Verilog)<br>~240 lines of Verilog for shadow logic<br>~8h manual effort for shadow logic |

**Table 1.** Detailed processor configurations.

## 6 Experiment Setup

We implement shadow logic for verifying the sandboxing contract and the constant-time contract (Section 2.2) on four processors as shown in Table 1. Other than the three open-source processors, SimpleOoO is a toy out-of-order processor developed by us that has the basic out-of-order execution behaviors, and we augment it with different defense mechanisms. In Table 1, we compare the code size for each processor that we verified and the code size and manual effort for

| | Sandboxing Contract | | | | | |
|---|---|---|---|---|---|---|
| | Sodor | Simple OoO-S | Simple OoO | Ridecore | BOOM | BOOM-S |
| Baseline | 🕒 | 🕒 | 🐞 | 🐞 | | |
| LEAVE [69] | ☺ | ⚠ | ⚠ | | | |
| UPEC [22, 36] | | | | | 🐞 | ☺ |
| Our Scheme | ☺ | ☺ | 🐞 | 🐞 | 🐞 | 🕒 |

**Table 2.** A summary of comparing the Baseline, LEAVE [69], UPEC [22, 36], and our scheme on verifying the sandboxing contract on six processor designs. "🐞" indicates valid attacks are found , "☺" indicates a proof is found, "🕒" indicates a time out on the verification task after running for 7 days, "⚠" indicates false counterexamples are found, and a shaded entry indicates the experiment is not conducted.

the shadow logic. Note that although BOOM is much larger than Ridecore, its shadow logic is less complex because it only commits one instruction every cycle while Ridecore is a superscalar processor that can commit two instructions per cycle. This shows that the complexity of our shadow logic does not necessarily grow as the processor gets larger.

Here is the overview of our verification workflow:

1. Implement the Contract Shadow Logic that extracts and compares the information from two out-of-order processors. The design under verification (DUV) includes both processors and the shadow logic.
2. Initialize the instruction memories in two out-of-order processors to have symbolic values, so all possible programs will be explored in model checking.
3. Write assumptions (constraints) including: (1) the two out-of-order processors have the same initial state except that the secret in memory has different initial values; (2) the contract constraint check must always hold.
4. Model check the DUV with the above assumptions and the assertion that leakage assertion check holds.

We perform verification tasks using the commercial model checking tool, JasperGold [10]. We use its `Mp` and `AM` solving engines to find proofs, and use the `Ht` engine to detect leakage. The performance results were obtained on a server with an Intel Xeon E5-4610 processor running at 2.4 GHz. We set 7 days as the time-out period for each verification task.

# 7 Experiment Results

## 7.1 Comparison with Baseline and Existing Schemes

To better understand how our work compares to the existing techniques, we conduct experiments to compare our scheme with the baseline (described in Section 4.1), LEAVE [69], and UPEC [22, 36]. We compare the verification results on six processor designs, three secure ones (`Sodor`, `SimpleOoO-S`, and `BOOM-S`) and three insecure ones (`SimpleOoO`, `Ridecore`, and `BOOM`), regarding two contracts, the sandboxing contract and the constant-time programming contract. The secure version of `SimpleOoO`, noted as `SimpleOoO-S`, delays the issue time of a memory instruction until its commit time if, at the time when it enters the pipeline, there is a branch before it in the ROB. The secure version of `BOOM`, noted as `BOOM-S`, delays the issue time of a memory instruction until it is guaranteed to commit.[2]

### 7.1.1 Comparison Summary.

The comparison results regarding the sandboxing contract are summarized in Table 2. We observe similar results regarding the constant-time contract except UPEC, which does not support it. Except for `BOOM-S`, our scheme can successfully find attacks or proofs within a reasonable amount of time (concrete numbers reported later).

- Compared to the baseline, our scheme has a clear advantage in conducting unbounded proofs, where the baseline easily times out on finding proofs on secure designs. We observe similar performance in finding attacks between the baseline and our scheme.
- Similar to our scheme, LEAVE can find proof on in-order processors (`Sodor`). More importantly, our scheme performs better than LEAVE on out-of-order processors. While LEAVE struggles with `SimpleOOO` designs, generating false counterexamples for both secure and insecure designs, our approach can find bugs and prove security of these designs. As discussed later, this can happen when insufficient invariants are used, and the induction procedure will consider unreachable states.
- UPEC is a verification scheme customized for the sandboxing contract and does not support the constant-time contract. It is able to find the same attacks on BOOM as our scheme but requires advanced formal-method expertise to manually derive invariants. We did not further adapt UPEC to other processors for comparison due to the amount of manual effort required.
- Our scheme still cannot scale to fully proving the security of large out-of-order processors such as `BOOM-S`.

We provide a detailed comparison below.

### 7.1.2 Comparison to Baseline.

When verifying a secure design (the first two columns), the baseline scheme times out. In contrast, our scheme can successfully find the proof within 151 minutes. In formal verification, a time out indicates that the model checker is hitting an exponential wall on the model being verified, i.e., the exponential size of the reachable state space is beyond the reach of the solver. This

[2]The time that a memory instruction becomes guaranteed to commit is slightly different between the `BOOM-S` evaluated in UPEC [22] and the one evaluated by us. Specifically, UPEC [22] applies the defense to a release version of Boom V2 and delays the issue time of a memory instruction until there is no unresolved branch ahead. We apply the defense to Boom V3 (a newer version) and delay memory instructions longer until they reach the head of ROB. We make this change to avoid Meltdown attacks that are re-introduced in Boom V3 [36].

result highlights that our scheme shows much better scalability of RTL verification for software-hardware contracts compared with the baseline scheme.

**7.1.3 Comparison with LEAVE.** LEAVE uses an inductive approach to verify processors against software-hardware contracts. Their key contribution is to automatically derive inductive invariants to assist deriving an inductive proof. These invariants are supposed to capture the reachable states of the state machines being verified. They start with a set of automatically generated or manually written candidate invariants and use a search algorithm to iteratively remove invariants that do not hold. The expectation is that the final remaining invariants are strong enough to prove security. Otherwise, when insufficient invariants are used, the induction procedure will consider *unreachable* states and generate false counterexamples. As such, they declare the verification result as *UNKNOWN*. An UNKNOWN result indicates that either the processor may be secure, but the search algorithm cannot find a good set of invariants to make the proof go through, or the processor may be insecure. As a side note, LEAVE proposes invariants that consider the instruction inclusion and synchronization requirements (Section 5.2) in a limited way for in-order processors. Our scheme is the first to satisfy these two requirements in a general way, supporting out-of-order and superscalar processors.

LEAVE has only been used to verify in-order processors. As LEAVE is open-sourced [68], we extend the evaluation of LEAVE to out-of-order processors. We use the automatically generated candidate invariants described in LEAVE's paper, i.e., values in corresponding registers are equivalent in the two copies of out-of-order processors (some of these may be dropped later).

When verifying an insecure version of `SimpleOoO`, our scheme finds an attack in 2 seconds. However, we observe LEAVE fails to find a set of invariants to prove security and states that the result is UNKNOWN, i.e., LEAVE cannot tell whether the design is secure or not. When verifying `SimpleOOO-S` (the secure version), our scheme is able to complete the proof within 2.5 hours, while LEAVE reports UNKNOWN after searching invariants for 2.1 hours. During its execution, we observe that LEAVE's invariant-search algorithm continuously removes invariants from the candidate sets until there are none left. The reason is that the automatically generated candidate invariants are insufficient to cover reachable states for even simple out-of-order processors. In these cases, LEAVE will require additional effort to improve the invariant generation algorithm or ask the user to manually write invariants, both posing additional challenges.

**7.1.4 Comparison with UPEC.** UPEC uses an inductive verification approach, assisted with substantial manual invariants provided by formal-method experts, to verify complex out-of-order processors including `BOOM`. In contrast to LEAVE which identifies invariants automatically, UPEC manually derives invariants through an iterative procedure. In each verification iteration, UPEC launches a verification task with the set of invariants. If a counterexample is found, the users manually inspect it to check whether it is due to a vulnerability or a lack of invariants. For the latter case, the users pinpoint the microarchitectural unit that exhibits an unreachable state in the execution trace and then leverage their understanding of the design to come up with more invariants to exclude this case. The iteration continues until a vulnerability or an unbounded proof is found.

In the above process, UPEC relies on the user's microarchitectural knowledge to guarantee the correctness of the invariants. If improper invariants are used, which may overconstrain the reachable states considered by the verifier, the verification result can be untrustworthy. Moreover, as discussed in Section 1, for the proof on `BOOM` to go through, UPEC uses a large number of invariants constraining the reachable states of reorder buffer, load/store queues, caches, floating point unit, ALU, etc., in total, requiring more than 20,000 lines of code.

To compare our scheme with UPEC, we implement our Contract Shadow Logic on `BOOM` and show this small amount of code (i.e., ~240 lines of Verilog as shown in Table 1) is able to find attacks on `BOOM` for both sandboxing and constant-time contracts. We use the `SmallBOOM` configuration (the release version is Boom V3) and directly use the data cache and instruction cache as the memory with the virtual memory module turned off. Both cache sizes are configured to 256 bytes. After adding the shadow logic, we can conduct the verification directly and no manual inspection is required to check the validity of counterexamples.

We find multiple attacks on `BOOM` exploiting different types of speculation. First, we find an attack for the sandboxing contract in 120 minutes and a similar attack for the constant-time contract in 36.6 minutes. In these two attacks, mis-speculation is triggered by exceptions due to memory address misalignment:

```
lui r6, 65536 //set addr to a legal range
lhu r14, 1221(r6) //mis-alignment in addr
lh r0, r14    //use secret as load addr
```

In the above code snippet, the second instruction loads the secret into `r14` and the third instruction uses the secret as memory address, thus violating the leakage assertion check. Since the second instruction triggers an exception due to address misalignment, the second and third instructions will not be committed, and thus will satisfy the contract constraint check.

Second, we can continue to search for other attacks following the standard practice in formal verification. We add an assumption to exclude the first attack that we found. The added assumption ensures the input program does not

| | Sandboxing | Constant-Time |
|---|---|---|
| $\text{NoFwd}_{\text{futuristic}}$ | 66min | 0.4s |
| $\text{NoFwd}_{\text{spectre}}$ | 45h | 0.1s |
| $\text{Delay}_{\text{futuristic}}$ | 21min | 10min |
| $\text{Delay}_{\text{spectre}}$ | 151min | 37min |
| $\text{DoM}_{\text{spectre}}$ | 6.5min[3] | 5.9min |

**Table 3.** Verification time using our scheme on `SimpleOoO` augmented with different defenses. The red entry indicates an attack is found and the green entry indicates a proof is found.

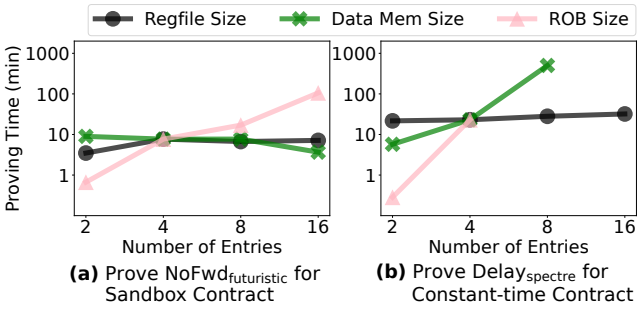


**Figure 2.** Verification time when varying the size of the register file, data memory, and re-order buffer.

involve memory accesses using misaligned addresses. Running the verification procedure for the sandboxing contract leads us to the discovery of a new attack exploiting exceptions due to illegal memory accesses. The verification for the constant-time contract generates an attack exploiting branch misprediction. The former case takes 8.7 hours, and the latter case takes 1.4 hours.

Finally, we further continue the experiments to exclude the two attacks. The verification tasks time out after 24 hours.

The above experiments show that, compared with UPEC, our scheme significantly reduces the amount of manual effort and can still find vulnerabilities in complex out-of-order processors (e.g., `BOOM`) for both sandboxing and constant-time contracts. We acknowledge that our scheme still cannot scale to fully proving the security of complex out-of-order processors (e.g., `BOOM-S`), which is possible for UPEC through substantial manual effort.

### 7.2 Impact of Defense Mechanisms

We now study how various factors can affect the verification time of our scheme. We start with the influence of defense mechanisms on verification time. We extend the `SimpleOoO` core with five microarchitecture defenses from the literature [52, 71, 77]. Note that we can directly reuse the shadow logic we developed for `SimpleOoO`, highlighting the reusability of our approach. We note this is the *first* time for many of these defense proposals to be evaluated as RTL designs, in contrast to high-level abstract models [28, 60, 76] or via gem5 simulation [9].

**Defenses** The following two mechanisms share similarities with the defenses described in STT [77] and NDA [71].

- $\text{NoFwd}_{\text{futuristic}}$: Do not forward the data of a load instruction to younger instructions until its commit time.
- $\text{NoFwd}_{\text{spectre}}$: Do not forward the data of a load instruction to younger instructions until its commit time, if when the load enters the pipeline, there is a branch before the load in the ROB.

The difference between the two mechanisms is that $\text{NoFwd}_{\text{futuristic}}$ considers all possible sources of speculation, while $\text{NoFwd}_{\text{spectre}}$ only considers branch prediction as the source of speculation.

The next three defenses work by delaying load instructions at different points in their lifetime.

- $\text{Delay}_{\text{futuristic}}$: Delay the issue time of a memory instruction until its commit time.
- $\text{Delay}_{\text{spectre}}$: Delay the issue time of a memory instruction until its commit time, if at the time when it enters the pipeline, there is a branch before it in the ROB. It is noted as `SimpleOoO-S` in Section 7.1.
- $\text{DoM}_{\text{spectre}}$: This models a simplified version of the Delay-on-Miss defense [52]. It always speculatively issues a load instruction. If the load hits the L1 cache, return the data to the core and proceed. If the load misses the L1 cache, delay the load from being sent to the main memory if at the time when the load enters the pipeline, there is a branch before the load in the ROB. We model a cache with a single cache entry with a 1-cycle hit and a 3-cycle miss.

$\text{Delay}_{\text{futuristic}}$ and $\text{Delay}_{\text{spectre}}$ are secure on `SimpleOoO` for both the sandboxing contract and constant-time contract, while $\text{DoM}_{\text{spectre}}$ is not secure. The vulnerability of $\text{DoM}_{\text{spectre}}$ has been discovered in prior work [6, 23].

**Evaluation Results** Table 3 compares the checking time of verifying the five defenses. For all the cases where we are supposed to find a proof, i.e., when the designs are secure, our scheme can find a proof for all of them within 45 hours, with most completing in about an hour. We note in all the cases, the baseline scheme times out (not shown in the table).

Comparing the checking time for different defenses, we observe that defenses have a non-trivial impact on checking time. First, it takes much longer to find the proof in secure designs (above 10 minutes) than to find an attack in insecure designs (below 10 minutes). Second, it takes less time to find a proof on a relatively more conservative defense mechanism. Comparing the pair of `NoFwd` and the pair of `Delay` defenses, the futuristic version takes much less time to verify.

[3]This attack is found using an 8-entry ROB instead of the default 4-entry ROB because it requires more concurrent instructions.

### 7.3 Impact of Structure Sizes

Finally, we study how structure sizes of re-order buffer, register file, and memory affect verification time. Fig. 2 shows the checking time for verifying $\mathsf{NoFwd}_{\mathsf{futuristic}}$ defense for the sandboxing contract and $\mathsf{Delay}_{\mathsf{spectre}}$ defense for the constant-time contract. The default configuration consistently uses 4 entries for the register file, ROB, and data memory. The three lines in the plot are derived by changing one of the three structures and keeping the other two unchanged.

Overall, the size of the register file has negligible impact on verification time. The size of data memory has a limited impact on proving the sandboxing contract but has a larger impact on proving the constant-time contract. Note that the size of ROB has a significant impact (y-axis is log scale). When we increase the ROB size, the verification time increases exponentially. The ROB size determines the number of inflight instructions, and the space of possible interactions between instructions grows quickly with ROB size, taking a much longer time to check.

## 8 Limitations and Future Work

Despite being able to achieve significant verification performance improvement compared to the baseline and several related works, we acknowledge this paper has several limitations. First, one appealing feature of our verification scheme is that the proposed contract shadow logic is *conceptually* reusable across different designs, given that the primary task of the shadow logic is to simply extract committed instruction information. We must concede that, thus far, constructing and synchronizing the shadow logic for the designs in our evaluation has relied on manual effort. To further reduce the required manual effort, we want to investigate how to partially automate this process. Our problem shares similarity to *tandem verification* [74] for functional correctness, where the same ISA-level information is extracted from RTL implementations. Applying techniques similar to [74] might help in automating part of the effort.

Second, our scheme does not fully address the scalability issue for verifying out-of-order processors with a realistic ROB size, which is usually above 100. This scalability issue is a well-known problem in hardware verification. On the positive side, we believe a reasonably reduced-size out-of-order machine already exhibits the essential security-relevant features of a real-size out-of-order machine, including speculation, resource contention, and instruction reordering. However, we believe work can be done to further improve the scalability. One promising direction is to explore taint propagation techniques like GLIFT [55, 57] in formal verification. This will avoid checking information leakage which is a hyperproperty, and thus needs two copies, to checking taint-propagation to observable outputs which is a safety property and thus needs only only a single copy, albeit with additional taint-propagation logic. As studied in prior work [30], taint propagation requires making a delicate trade-off between precision and verification overhead. The taint propagation logic can be customized to the security property and specific hardware design to accomplish this. We hypothesize that taint propagation techniques could be combined with our scheme to further improve verification performance.

Finally, as we explore the relationship between manually crafted invariants and and the information embedded within our shadow logic, some additional insights emerge. We speculate that our shadow logic may offer a rich array of invariants to the model checker. This hypothesis is rooted in the notion that contract constraints, originally designed to constrain software programs, indirectly strongly constrain the set of reachable states of the processor. However, more investigation is needed to understand the constraining power of the shadow logic and how this influences the performance of the verification procedure.

## 9 Related Work

Among all related works, UPEC [21, 22, 36] and LEAVE [69] are the closest to us, as they both target speculative execution vulnerabilities and work on RTL. We have provided detailed comparisons of our scheme, UPEC, and LEAVE in Section 7.1. We now discuss other related work. We identify that previous approaches differ along three axes: 1) verification target, 2) security properties, and 3) checking techniques.

**Targets for Verification of Secure Speculation** There is a lot of work leveraging formal methods to verify secure speculation properties (variants of software-hardware contracts), but their targets differ from our work, either focusing on abstract processor models or software.

Pensieve [76], Guarnieri et al. [28], and CheckMate [60] target abstract processor models (not RTL). Specifically, Pensieve [76] does bounded model checking to detect speculative execution vulnerabilities on modular models connected using hand-shaking interfaces. Guarnieri et al. [28] uses a paper proof to reason about an abstract out-of-order processor model expressed as operational semantics. CheckMate [60] applies security litmus tests on abstract processor models expressed as $\mu$hb graphs to synthesize exploit patterns, so as to find new attacks through these patterns. Although verifying abstract processor models can give designers key insights, this approach unavoidably neglects RTL implementation details, which may cause speculative execution attacks.

On the software side, there are efforts to guarantee that a program does not contain gadgets that can be exploited by different speculative execution attacks [13, 16, 47, 48, 66, 72]. For example, Serberus [48] detects code patterns, at compile time, in cryptographic primitives that can be exploited by broad ranges of speculative execution attacks. This line of work usually assumes a specific hardware model, and then verifies that specific pieces of software satisfy the software constraint component in software-hardware contracts. The

goal is very different from our work, where we verify the hardware's RTL implementation.

**Security Properties** There exist RTL security verification works that target security properties *not* related to speculative execution. These works target various data-oblivious properties. For example, Clepsydra [4] verifies hardware modules, such as ALUs and Caches, take the same amount of time when executing on different input data. IODINE [25], Xenon [61], UPEC-DIT [20] check whether each instruction's execution time is independent of its operands for in-order/out-of-order processors. Knox [5] builds a framework to verify that when a specific program is running on a physically-isolated Hardware Security Module (HSM), its execution time does not leak extra information beyond the functional specification of the program.

**Checking Techniques** In addition to leveraging formal-method techniques, fuzzing-based schemes [24, 32, 46, 49, 50, 70, 73], such as SpecDoctor [32], check RTL or blackbox hardware for speculative execution vulnerabilities by generating and testing random attack patterns. Although the above works cannot guarantee full coverage of speculative execution attacks, they can be faster at detecting security bugs when evaluating large designs.

**Verification of Clean-slate Designs** Finally, an exciting yet orthogonal line of work focuses on security verification of clean-slate designs. SecVerilog [79] enables hardware programmers to enforce security policies while developing the hardware. A violation of the security policy would be detected at compile time. SpecVerilog [78] further extends this ability to support specifying speculation-related policies. [56] formally verifies that a low-trust architecture with a specialized ISA and hardware is free of timing side-channel leakage.

## 10 Conclusions

Our paper presents Contract Shadow Logic, a broadly-applicable approach for performing formal verification of security properties on RTL designs for software-hardware contracts. Our scheme asks processor designers to construct shadow logic to extract ISA observation traces from processors and enforce the instruction inclusion and synchronization requirements. Our evaluation shows the strength of our scheme compared with the baseline and two state-of-the-art RTL verification schemes in both finding attacks on insecure designs and deriving unbounded proofs for secure designs.

## 11 Acknowledgment

This work was supported in part by NSF under grants CCF 2217099 and CNS 2046359; by ACE, one of the seven centers in JUMP 2.0, a Semiconductor Research Corporation (SRC) program sponsored by DARPA; by the Swiss State Secretariat for Education, Research, and Innovation (SERI) through the SwissChips research project.

## References

[1] Sam Ainsworth. Ghostminion: A strictness-ordered cache system for spectre mitigation. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, pages 592–606, 2021.

[2] Sam Ainsworth and Timothy M. Jones. Muontrap: Preventing cross-domain spectre-like attacks by capturing speculative state. In *Proceedings of the ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020.

[3] AMD. Software techniques for managing speculation on amd processors. https://www.amd.com/system/files/documents/software-techniques-for-managing-speculation.pdf, 2023. [Accessed 23-06-2024].

[4] Armaiti Ardeshiricham, Wei Hu, and Ryan Kastner. Clepsydra: Modeling timing flows in hardware designs. In *2017 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 147–154. IEEE, 2017.

[5] Anish Athalye, M Frans Kaashoek, and Nickolai Zeldovich. Verifying hardware security modules with {Information-Preserving} refinement. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, pages 503–519, 2022.

[6] Mohammad Behnia, Prateek Sahu, Riccardo Paccagnella, Jiyong Yu, Zirui Neil Zhao, Xiang Zou, Thomas Unterluggauer, Josep Torrellas, Carlos Rozas, Adam Morrison, et al. Speculative interference attacks: Breaking invisible speculation schemes. In *Proceedings of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 1046–1060, 2021.

[7] Sergey Berezin, Edmund Clarke, Armin Biere, and Yunshan Zhu. Verification of out-of-order processor designs using model checking and a light-weight completion function. *Formal Methods in System Design*, 20:159–186, 2002.

[8] Atri Bhattacharyya, Alexandra Sandulescu, Matthias Neugschwandtner, Alessandro Sorniotti, Babak Falsafi, Mathias Payer, and Anil Kurmus. Smotherspectre: exploiting speculative execution through port contention. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 785–800, 2019.

[9] Nathan Binkert, Bradford Beckmann, Gabriel Black, Steven K Reinhardt, Ali Saidi, Arkaprava Basu, Joel Hestness, Derek R Hower, Tushar Krishna, Somayeh Sardashti, Rathijit Sen, Korey Sewell, Muhammad Shoaib, Nilay Vaish, Mark D. Hill, and David A. Wood. The Gem5 Simulator. *ACM SIGARCH Computer Architecture News*, 2011.

[10] Cadence. Jasper Formal Property Verification App — cadence.com. https://www.cadence.com/en_US/home/tools/system-design-and-verification/formal-and-static-verification/jasper-gold-verification-platform/formal-property-verification-app.html. [Accessed 23-06-2024].

[11] Claudio Canella, Daniel Genkin, Lukas Giner, Daniel Gruss, Moritz Lipp, Marina Minkin, Daniel Moghimi, Frank Piessens, Michael Schwarz, Berk Sunar, et al. Fallout: Leaking data on meltdown-resistant cpus. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, 2019.

[12] Chandler Carruth. Speculative Load Hardening. https://llvm.org/docs/SpeculativeLoadHardening.html, 2018. [Accessed 23-06-2024].

[13] Sunjay Cauligi, Craig Disselkoen, Klaus v Gleissenthall, Dean Tullsen, Deian Stefan, Tamara Rezk, and Gilles Barthe. Constant-time foundations for the new spectre era. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 913–926, 2020.

[14] Sunjay Cauligi, Craig Disselkoen, Daniel Moghimi, Gilles Barthe, and Deian Stefan. SoK: Practical foundations for software spectre defenses. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 666–680. IEEE, 2022.

[15] Christopher Celio. RISCV-Sodor. https://github.com/ucb-bar/riscv-sodor, 2013. [Accessed 23-06-2024].

[16] Kevin Cheang, Cameron Rasmussen, Sanjit Seshia, and Pramod Subramanyan. A formal approach to secure speculation. In *2019 IEEE 32nd Computer Security Foundations Symposium (CSF)*, pages 288–28815. IEEE, 2019.

[17] Md Hafizul Islam Chowdhuryy and Fan Yao. Leaking secrets through modern branch predictors in the speculative world. *IEEE Transactions on Computers*, 71(9):2059–2072, 2021.

[18] Michael R Clarkson and Fred B Schneider. Hyperproperties. *Journal of Computer Security*, 18(6):1157–1210, 2010.

[19] Dask. Dask: a python library for parallel and distributed computing. https://dask.org, 2024. Accessed 8 July 2024.

[20] Lucas Deutschmann, Johannes Mueller, Mohammad Rahmani Fadiheh, Dominik Stoffel, and Wolfgang Kunz. A scalable formal verification methodology for data-oblivious hardware. *arXiv preprint arXiv:2308.07757*, 2023.

[21] Mohammad Rahmani Fadiheh, Johannes Müller, Raik Brinkmann, Subhasish Mitra, Dominik Stoffel, and Wolfgang Kunz. A formal approach for detecting vulnerabilities to transient execution attacks in out-of-order processors. In *2020 57th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2020.

[22] Mohammad Rahmani Fadiheh, Alex Wezel, Johannes Müller, Jörg Bormann, Sayak Ray, Jason M Fung, Subhasish Mitra, Dominik Stoffel, and Wolfgang Kunz. An exhaustive approach to detecting transient execution side channels in RTL designs of processors. *IEEE Transactions on Computers*, 72(1):222–235, 2022.

[23] Jacob Fustos, Michael Bechtel, and Heechul Yun. Spectrerewind: Leaking secrets to past instructions. In *Proceedings of the 4th ACM Workshop on Attacks and Solutions in Hardware Security*. ACM, 2020.

[24] Moein Ghaniyoun, Kristin Barber, Yinqian Zhang, and Radu Teodorescu. Introspectre: A pre-silicon framework for discovery and analysis of transient execution vulnerabilities. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, pages 874–887. IEEE, 2021.

[25] Klaus v Gleissenthall, Rami Gökhan Kıcı, Deian Stefan, and Ranjit Jhala. IODINE: Verifying Constant-Time execution of hardware. In *28th USENIX Security Symposium (USENIX Security 19)*, pages 1411–1428, 2019.

[26] Aman Goel and Karem Sakallah. AVR: abstractly verifying reachability. In *Tools and Algorithms for the Construction and Analysis of Systems: 26th International Conference, TACAS 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Dublin, Ireland, April 25–30, 2020, Proceedings, Part I 26*, pages 413–422. Springer, 2020.

[27] Ben Gras, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. Translation leak-aside buffer: Defeating cache side-channel protections with TLB attacks. In *27th USENIX Security Symposium (USENIX Security 18)*, pages 955–972, 2018.

[28] Marco Guarnieri, Boris Köpf, Jan Reineke, and Pepe Vila. Hardware-software contracts for secure speculation. In *2021 IEEE Symposium on Security and Privacy (SP)*, pages 1868–1883. IEEE, 2021.

[29] Jann Horn. Speculative execution, variant 4: Speculative store bypass. https://bugs.chromium.org/p/project-zero/issues/detail?id=1528, 2018. [Accessed 23-06-2024].

[30] Wei Hu, Jason Oberg, Ali Irturk, Mohit Tiwari, Timothy Sherwood, Dejun Mu, and Ryan Kastner. Theoretical fundamentals of gate level information flow tracking. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 30(8):1128–1140, 2011.

[31] Bo-Yuan Huang, Hongce Zhang, Pramod Subramanyan, Yakir Vizel, Aarti Gupta, and Sharad Malik. Instruction-level abstraction (ILA): A uniform specification for system-on-chip (SoC) verification. *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, 24(1):1–24, 2018.

[32] Jaewon Hur, Suhwan Song, Sunwoo Kim, and Byoungyoung Lee. Specdoctor: Differential fuzz testing to find transient execution vulnerabilities. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security*, pages 1473–1487, 2022.

[33] Arch Lab. in Tokyo Institute of Technology. RIsc-v Dynamic Execution CORE. https://github.com/ridecore/ridecore, 2016. [Accessed 23-06-2024].

[34] Intel. Intel Side Channel Vulnerability L1TF. https://www.intel.com/content/www/us/en/architecture-and-technology/l1tf.html, 2018. [Accessed 23-06-2024].

[35] Intel. Intel Analysis of Speculative Execution Side Channels. https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/technical-documentation/analysis-speculative-execution-side-channels.html, 2021. [Accessed 23-06-2024].

[36] Tobias Jauch, Alex Wezel, Mohammad R Fadiheh, Philipp Schmitz, Sayak Ray, Jason M Fung, Christopher W Fletcher, Dominik Stoffel, and Wolfgang Kunz. Secure-by-construction design methodology for cpus: Implementing secure speculation on the rtl. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–9. IEEE, 2023.

[37] Khaled N Khasawneh, Esmaeil Mohammadian Koruyeh, Chengyu Song, Dmitry Evtyushkin, Dmitry Ponomarev, and Nael Abu-Ghazaleh. Safespec: Banishing the spectre of a meltdown with leakage-free speculation. In *2019 56th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6. IEEE, 2019.

[38] Vladimir Kiriansky and Carl Waldspurger. Speculative buffer overflows: Attacks and defenses. *arXiv preprint arXiv:1807.03757*, 2018.

[39] Paul Kocher, Jann Horn, Anders Fogh, , Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. Spectre attacks: Exploiting speculative execution. In *40th IEEE Symposium on Security and Privacy (S&P'19)*, 2019.

[40] Esmaeil Mohammadian Koruyeh, Khaled N Khasawneh, Chengyu Song, and Nael Abu-Ghazaleh. Spectre returns! speculation attacks using the return stack buffer. In *12th USENIX Workshop on Offensive Technologies (WOOT 18)*, 2018.

[41] K Rustan M Leino. Dafny: An automatic program verifier for functional correctness. In *International conference on logic for programming artificial intelligence and reasoning*, pages 348–370. Springer, 2010.

[42] Moritz Lipp, Michael Schwarz, Daniel Gruss, Thomas Prescher, Werner Haas, Anders Fogh, Jann Horn, Stefan Mangard, Paul Kocher, Daniel Genkin, Yuval Yarom, and Mike Hamburg. Meltdown: Reading kernel memory from user space. In *27th USENIX Security Symposium (USENIX Security 18)*, 2018.

[43] Kevin Loughlin, Ian Neal, Jiacheng Ma, Elisa Tsai, Ofir Weisse, Satish Narayanasamy, and Baris Kasikci. DOLMA: Securing speculation with the principle of transient Non-Observability. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 1397–1414, 2021.

[44] Giorgi Maisuradze and Christian Rossow. ret2spec: Speculative execution using return stack buffers. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, 2018.

[45] Stefan Mangard, Elisabeth Oswald, and Thomas Popp. *Power Analysis Attacks: Revealing the Secrets of Smart Cards (Advances in Information Security)*. Springer-Verlag, 2007.

[46] Daniel Moghimi, Moritz Lipp, Berk Sunar, and Michael Schwarz. Medusa: Microarchitectural data leakage via automated attack synthesis. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 1427–1444, 2020.

[47] Nicholas Mosier, Hanna Lachnitt, Hamed Nemati, and Caroline Trippel. Axiomatic hardware-software contracts for security. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, pages 72–86, 2022.

[48] Nicholas Mosier, Hamed Nemati, John C Mitchell, and Caroline Trippel. Serberus: Protecting cryptographic code from spectres at compile-time. *arXiv preprint arXiv:2309.05174*, 2023.

[49] Oleksii Oleksenko, Christof Fetzer, Boris Köpf, and Mark Silberstein. Revizor: Testing black-box cpus against speculation contracts. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 226–239, 2022.

[50] Oleksii Oleksenko, Marco Guarnieri, Boris Köpf, and Mark Silberstein. Hide and seek with spectres: Efficient discovery of speculative information leaks with random testing. *arXiv preprint arXiv:2301.07642*, 2023.

[51] Alastair Reid, Rick Chen, Anastasios Deligiannis, David Gilday, David Hoyes, Will Keen, Ashan Pathirane, Owen Shepherd, Peter Vrabel, and Ali Zaidi. End-to-end verification of processors with isa-formal. In *Computer Aided Verification: 28th International Conference, CAV 2016, Toronto, ON, Canada, July 17-23, 2016, Proceedings, Part II 28*, pages 42–58. Springer, 2016.

[52] Christos Sakalis, Stefanos Kaxiras, Alberto Ros, Alexandra Jimborean, and Magnus Själander. Efficient invisible speculative execution through selective delay and value prediction. In *Proceedings of the 46th International Symposium on Computer Architecture*, pages 723–735, 2019.

[53] Philipp Schmitz, Johannes Mueller, Christian Bartsch, Dominik Stoffel, and Wolfgang Kunz. Upec-pn: Exhaustive constant time verification of low-level software using property checking. In *MBMV 2023; 26th Workshop*, pages 1–8. VDE, 2023.

[54] Michael Schwarz, Moritz Lipp, Daniel Moghimi, Jo Van Bulck, Julian Stecklina, Thomas Prescher, and Daniel Gruss. Zombieload: Cross-privilege-boundary data sampling. In *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security*, pages 753–768, 2019.

[55] Flavien Solt, Ben Gras, and Kaveh Razavi. CellIFT: Leveraging cells for scalable and precise dynamic information flow tracking in RTL. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 2549–2566, 2022.

[56] Qinhan Tan, Yonathan Fisseha, Shibo Chen, Lauren Biernacki, Jean-Baptiste Jeannin, Sharad Malik, and Todd Austin. Security verification of low-trust architectures. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 945–959, 2023.

[57] Mohit Tiwari, Hassan MG Wassel, Bita Mazloom, Shashidhar Mysore, Frederic T Chong, and Timothy Sherwood. Complete information flow tracking from the gates up. In *Proceedings of the 14th international conference on Architectural support for programming languages and operating systems*, pages 109–120, 2009.

[58] Youssef Tobah, Andrew Kwong, Ingab Kang, Daniel Genkin, and Kang G Shin. Spechammer: Combining spectre and rowhammer for new speculative attacks. In *2022 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2022.

[59] Alex Wezel Tobias Jauch. UPEC Verification Suite. https://github.com/RPTU-EIS/SecureBOOM. [Accessed 12-07-2024].

[60] Caroline Trippel, Daniel Lustig, and Margaret Martonosi. Checkmate: Automated synthesis of hardware exploits and security litmus tests. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 947–960. IEEE, 2018.

[61] Klaus v. Gleissenthall, Rami Gökhan Kıcı, Deian Stefan, and Ranjit Jhala. Solver-aided constant-time hardware verification. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, pages 429–444, 2021.

[62] Jo Van Bulck, Marina Minkin, Ofir Weisse, Daniel Genkin, Baris Kasikci, Frank Piessens, Mark Silberstein, Thomas F Wenisch, Yuval Yarom, and Raoul Strackx. Foreshadow: Extracting the keys to the intel SGX kingdom with transient Out-of-Order execution. In *27th USENIX Security Symposium (USENIX Security 18)*, pages 991–1008, 2018.

[63] Jo Van Bulck, Daniel Moghimi, Michael Schwarz, Moritz Lippi, Marina Minkin, Daniel Genkin, Yuval Yarom, Berk Sunar, Daniel Gruss, and Frank Piessens. LVI: Hijacking transient execution through microarchitectural load value injection. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 54–72. IEEE, 2020.

[64] Stephan Van Schaik, Alyssa Milburn, Sebastian Österlund, Pietro Frigo, Giorgi Maisuradze, Kaveh Razavi, Herbert Bos, and Cristiano Giuffrida. RIDL: Rogue in-flight data load. In *2019 IEEE Symposium on Security and Privacy (SP)*, pages 88–105. IEEE, 2019.

[65] Jose Rodrigo Sanchez Vicarte, Pradyumna Shome, Nandeeka Nayak, Caroline Trippel, Adam Morrison, David Kohlbrenner, and Christopher W Fletcher. Opening pandora's box: A systematic study of new ways microarchitecture can leak private data. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021.

[66] Guanhua Wang, Sudipta Chattopadhyay, Ivan Gotovchits, Tulika Mitra, and Abhik Roychoudhury. oo7: Low-overhead defense against spectre attacks via program analysis. *IEEE Transactions on Software Engineering*, 47(11):2504–2519, 2019.

[67] Yingchen Wang, Riccardo Paccagnella, Elizabeth Tang He, Hovav Shacham, Christopher W. Fletcher, and David Kohlbrenner. Hertzbleed: Turning power Side-Channel attacks into remote timing attacks on x86. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 2022.

[68] Zilong Wang, Gideon Mohr, Klaus von Gleissenthall, Jan Reineke, and Marco Guarnieri. Open-sourced framework of "specification and verification of side-channel security for open-source processors via leakage contracts". https://github.com/zilongwang123/LeaVe, 2023. Accessed 15 January 2024.

[69] Zilong Wang, Gideon Mohr, Klaus von Gleissenthall, Jan Reineke, and Marco Guarnieri. Specification and verification of side-channel security for open-source processors via leakage contracts. In *Proceedings of the 30th ACM Conference on Computer and Communications Security*, CCS 2023. ACM, 2023.

[70] Daniel Weber, Ahmad Ibrahim, Hamed Nemati, Michael Schwarz, and Christian Rossow. Osiris: Automated discovery of microarchitectural side channels. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 1415–1432, 2021.

[71] Ofir Weisse, Ian Neal, Kevin Loughlin, Thomas F Wenisch, and Baris Kasikci. NDA: Preventing speculative execution attacks at their source. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*, pages 572–586, 2019.

[72] Meng Wu and Chao Wang. Abstract interpretation under speculative execution. In *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation*, pages 802–815, 2019.

[73] Yuan Xiao, Yinqian Zhang, and Radu Teodorescu. Speechminer: A framework for investigating and measuring speculative execution vulnerabilities. *arXiv preprint arXiv:1912.00329*, 2019.

[74] Yue Xing, Aarti Gupta, and Sharad Malik. Generalizing tandem simulation: Connecting high-level and rtl simulation models. In *2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2022.

[75] Mengjia Yan, Jiho Choi, Dimitrios Skarlatos, Adam Morrison, Christopher Fletcher, and Josep Torrellas. Invisispec: Making speculative execution invisible in the cache hierarchy. In *2018 51st Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pages 428–441. IEEE, 2018.

[76] Yuheng Yang, Thomas Bourgeat, Stella Lau, and Mengjia Yan. Pensieve: Microarchitectural modeling for security evaluation. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*, pages 1–15, 2023.

[77] Jiyong Yu, Mengjia Yan, Artem Khyzha, Adam Morrison, Josep Torrellas, and Christopher W. Fletcher. Speculative Taint Tracking (STT): A

Comprehensive Protection for Speculatively Accessed Data. In *Proceedings of the 52nd Annual IEEE/ACM International Symposium on Microarchitecture*. ACM, 2019.

[78] Drew Zagieboylo, Charles Sherk, Andrew C Myers, and G Edward Suh. Specverilog: Adapting information flow control for secure speculation. In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security*, pages 2068–2082, 2023.

[79] Danfeng Zhang, Yao Wang, G Edward Suh, and Andrew C Myers. A hardware design language for timing-sensitive information-flow security. *Acm Sigplan Notices*, 50(4):503–516, 2015.

[80] Jerry Zhao, Ben Korpan, Abraham Gonzalez, and Krste Asanovic. Sonicboom: The 3rd generation berkeley out-of-order machine. May 2020.

# A Artifact Appendix

## A.1 Abstract

Our artifact is the infrastructure for verifying the two software-hardware contracts on four processors listed in Table 1. It includes the baseline verification scheme using four copies of processors and our verification scheme using contract shadow logic. This artifact can be used to reproduce the verification results, including the time it takes and the attacks it finds, shown in Table 2, Table 3, and Fig. 2.

## A.2 Artifact check-list (meta-information)

- **Run-time environment:** JasperGold and Dask.
- **Hardware:** A >16-core machine with 4GB memory per core.
- **Output:** Terminal outputs and a figure. Example results are included.
- **Experiments:** Running the provided scripts will reproduce the results. Results will vary due to the performance of the machine and the randomness in JasperGold's solving engines.
- **How much disk space required (approximately)?:** 1GB.
- **How much time is needed to prepare workflow (approximately)?:** 20 minutes.
- **How much time is needed to complete experiments (approximately)?:** On a >16-core machine, it takes ~3 hours to complete most (~42 out of 54) experiments and 7 days to complete all experiments.
- **Publicly available?:** Yes.
- **Code licenses (if publicly available)?:** MIT license.
- **Archived (provide DOI)?:** `10.5281/zenodo.14324584`

## A.3 Description

**A.3.1 How to access.** https://github.com/qinhant/ShadowLogicArtifact.

**A.3.2 Hardware dependencies.** The experiments only require CPU and memory resources. When running the experiment of a single configuration, it will take ~1.2 cores. We provide scripts to run experiments of many configurations in parallel and thus recommend a >16-core machine to finish them in a shorter time. 4GB memory per core is recommended to avoid out-of-memory errors.

**A.3.3 Software dependencies.** JasperGold FPV by Cadence [10] is required to run the experiments. It is a commercial hardware verification tool that requires a license to use. Our scripts assume JasperGold can be launched through `jg` commands from the terminal. Dask [19] is used to launch multiple experiments in parallel, it can be installed with `python3 -m pip install "dask[complete]"`.

## A.4 Installation

Download the artifact folder to the machine and make two copies of it, which will be used to launch two batches of experiments. No other installation is required.

## A.5 Experiment workflow

This artifact is used to reproduce the verification results on various tasks shown in Table 2, Table 3, and Fig. 2.

For each verification task, the verilog source code is located in the `src` folder, including four processors (`Sodor`, `Simple00O`, `Ridecore`, and `BOOM`). Each processor has its corresponding top module to conduct baseline verification scheme and our verification scheme using contract shadow logic under both sandboxing and constant-time contracts. The JasperGold scripts to launch each verification task are located in the `verification` folder.

The users of this artifact can use scripts provided in the `results` folder to launch batches of verification tasks. When the batches are running, the user can check the results of finished tasks by using the provided scripts to parse the log files and print out a summary of all verification results. A plotting script is provided to reproduce Fig. 2.

## A.6 Evaluation and expected results

**Reproduce Table 2 and Table 3** In the first copy of the artifact folder, we first initialize a cluster of 12 worker threads with:

```
(python3 -u scripts/dask/initCluster.py 12 \
         > & initCluster.log &)
```

Then, launch the batch of verification tasks with:

```
(python3 -u results/CompareTable/run.py \
         > & run.log &)
```

When the verification tasks are running, they will produce `my_proj_*` folders as JasperGold's project folder, `terminal_*.log` files as JasperGold's terminal output files, and `my_jdb_*` files saving the attacks found. Users can run the following commands to parse JasperGold's terminal output files and print out a summary of verification results on finished tasks:

```
python3 results/CompareTable/summary.py
python3 results/CompareTable/print.py
```

The commands can be repeated as more tasks are finished.

The summary is listed processor-by-processor. `results/CompareTable/example.log` is an example of it. For each processor, there will be three arrays:

- The first array is the names of the verification tasks with the following naming convention: `sandbox` and `ct` represents the sandboxing and constant-time contracts. `4copy` and `2copy` represents the baseline verification scheme and our scheme. For the `SimpleOoO`, the array includes six versions of the processor, including the insecure version with no defense scheme followed by the five versions with defense augmentation listed in Table 3.
- The second array indicates whether the processor under test is secure. `1` indicates the processor is secure, `0` indicates an attack is found, and `-1` indicates the experiment has not finished. You could further open JasperGold's GUI to check the attacks found with:

```
jg -command "restore -jdb my_jdb_{expName}"
```

- The third array includes the time it takes for the verification tasks to finish in the unit of seconds. `-1.0` indicates the experiment has not finished.

It takes ~3 hours to complete most (~25 out of 34) tasks and 7 days to complete all experiments. The summary is used to fill in the information in Table 2 and Table 3. Be aware that the results will vary due to the performance of the machine and the randomness in JasperGold's solving engines.

**Reproduce Fig. 2** In this batch of experiments, we can share the cluster created for the last batch. To launch the experiments, move to the second copy of the artifact folder and use similar commands as before:

```
(python3 -u results/ScalabilityFigure/run.py \
         > & run.log &)
## Wait ~3 hours to see most results.
python3 results/ScalabilityFigure/summary.py
python3 results/ScalabilityFigure/print.py
## Example print-outs in `example.log`.
```

The naming convention in the summary further includes `RF*_MEMI*_MEMD*_ROB*` to indicate the size of the register file, instruction memory, data memory, and re-order buffer. These parameters are scaled up to demonstrate how verification time will increase in Fig. 2.

To reproduce the figure, we provide a template script in `results/ScalabilityFigure/plot.py`. The array `time` in it stores the verification time to be plotted. The two sub-arrays in it correspond to the sandboxing and constant-time contracts. Each sub-array contains verification time when varying register, data memory, and re-order buffer sizes among 2, 4, 8, and 16. Carefully update the array `time` in it based on the experiment summary (be aware that 3 data points should be omitted because they cannot finished within 7 days in our machine setup) and plot with:

```
python3 results/ScalabilityFigure/plot.py
```

Finally, terminate the cluster by finding the `PID` of the thread and kill it:

```
ps -ef | grep initCluster.py
kill XXXX
```

### A.7 Methodology

Submission, reviewing and badging methodology:

- https://www.acm.org/publications/policies/artifact-review-badging
- http://cTuning.org/ae/submission-20201122.html
- http://cTuning.org/ae/reviewing-20201122.html